

Lab Exercise 6- Implementing API Rate Limiting (10 Requests per Minute)

1. Objective

The objective of this lab is to:

- Implement **API rate limiting (10 requests per minute)**
 - Restrict excessive API usage
 - Block clients exceeding the request threshold
 - Automatically **reset the limit after 1 minute**
 - Test API behavior using Postman
-

2. Software Requirements

Tool	Purpose
Java JDK 17	Programming
VS Code	Development
Maven	Build tool
Spring Boot	REST API framework
Postman	API testing

3. Create Spring Boot Project

Configuration

Group

com.example

Artifact

rate-limit-api

Dependencies

Spring Web

Spring Boot DevTools

Lombok

Download and open the project in **VS Code**.

4. Project Folder Structure

Create the following packages.

com.example

controller

ApiController.java

service

RateLimiterService.java

RateLimitApiApplication.java

pom.xml

```
<dependencies>
  <!-- Spring Boot Web (REST API) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Lombok (Optional but included as per lab requirement) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- DevTools for auto restart -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <!-- Testing -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

</dependencies>
```

5. Main Spring Boot Application

File

RateLimitApiApplication.java

```
package com.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RateLimitApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(RateLimitApiApplication.class, args);
    }
}
```

Explanation

This file **starts the Spring Boot application.**

6. Implement Rate Limiter Service (10 Requests per Minute)

Create file

service/RateLimiterService.java

Complete Code

```
package com.example.rate_limit_api.service;
import java.util.concurrent.ConcurrentHashMap;

import org.springframework.stereotype.Service;

@Service
public class RateLimiterService {

    private static final int MAX_REQUESTS_PER_MINUTE = 10;
    private static final long TIME_WINDOW = 60000; // 1 minute

    private ConcurrentHashMap<String, Integer> requestCounts = new
ConcurrentHashMap<>();
    private ConcurrentHashMap<String, Long> windowStart = new
ConcurrentHashMap<>();

    public synchronized boolean allowRequest(String ip) {

        long currentTime = System.currentTimeMillis();

        windowStart.putIfAbsent(ip, currentTime);
        requestCounts.putIfAbsent(ip, 0);

        long startTime = windowStart.get(ip);

        if ((currentTime - startTime) < TIME_WINDOW) {
```

```
int count = requestCounts.get(ip) + 1;
requestCounts.put(ip, count);

if (count > MAX_REQUESTS_PER_MINUTE) {
    return false;
}

} else {

    windowStart.put(ip, currentTime);
    requestCounts.put(ip, 1);
}

return true;
}
}
```

Explanation

This service:

- Tracks request count per IP
 - Allows **10 requests within a 60-second window**
 - Resets the counter after the minute expires
-

7. Create API Controller

File

controller/ApiController.java

Complete Code

```
package com.example.rate_limit_api.controller;
import com.example.rate_limit_api.service.RateLimiterService;

import jakarta.servlet.http.HttpServletRequest;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class ApiController {

    private final RateLimiterService rateLimiter;

    public ApiController(RateLimiterService rateLimiter) {
        this.rateLimiter = rateLimiter;
    }

    @GetMapping("/data")
    public String getData(HttpServletRequest request) {
        String clientIp = request.getRemoteAddr();

        if (!rateLimiter.allowRequest(clientIp)) {
            return "Rate limit exceeded. Maximum 10 requests per minute allowed.";
        }
        return "Secure API response";
    }
}
```

Explanation

The controller:

- Retrieves the **client IP address**
 - Checks if the request is allowed
 - Blocks the request if the limit is exceeded
-

8. Run the Application

Open VS Code terminal.

Run

```
mvn spring-boot:run
```

Server runs at

http://localhost:8080

9. Testing Using Postman

Open **Postman**.

Test 1 — Normal Request

Request

GET

http://localhost:8080/api/data

Response

Secure API response

Test 2 — Send 10 Requests

Send the request **10 times within one minute**.

Response

Secure API response

Test 3 — 11th Request

Send the request again.

Response

Rate limit exceeded. Maximum 10 requests per minute allowed.

Test 4 — Wait 1 Minute

After **60 seconds**, send the request again.

Response

Secure API response

This confirms that the **rate limit window has reset**.

10. Expected Behavior

Request Count	Response
1–10 requests	Allowed
11th request	Blocked
After 60 seconds	Allowed again

11. Security Benefits

Rate limiting protects APIs from:

Security Threat	Explanation
Brute force login attempts	Limits authentication requests
API abuse	Prevents automated scripts
Denial of Service	Reduces server overload

12. Learning Outcomes

After completing this lab, students will understand:

- How **rate limiting works in secure APIs**
- How to **track requests per client IP**
- How to **implement time-window rate limiting**
- How to **test API security mechanisms using Postman**